

Aviation News Collector — Node-by-Node Guide

Bu döküman, n8n workflow'undaki her node'un ne yaptığını, mevcut sorunları ve geliştirme önerilerini açıklar.

Genel Akış



Her Node'un Detaylı Açıklaması

1. Every 6h — Schedule Trigger

Her 6 saatte bir workflow'u otomatik başlatır. Şu an `active: false` olduğu için çalışmıyor.

Düzeltilme: Workflow'u publish/active yapınca bu tetikleyici devreye girer.

2. Run Manually — Manual Trigger

n8n arayüzünden elle "Execute" butonuna basıldığında başlatır. Test için kullanılır.

3. Load Sources — Code Node

⚠ **En kritik node.** Tüm kaynak listesi burada JavaScript kodu içinde hardcoded yazılmış.

```
const sources = [  
  { id: 'avherald', name: 'AVHerald', type: 'html', url: 'https://avherald.com', selector: '' },  
  ...  
];
```

Sorun: Kaynakları eklemek/silmek için bu node'u manuel düzenlemek gerekiyor.

Çözüm: MongoDB'deki `sources` collection'ından çekecek şekilde değiştirmek gerekir:

```
// Önerilen: MongoDB'den oku  
// n8n'de MongoDB node → Load Sources şeklinde bağla
```

Bu sayede Next.js sitesindeki Sources sayfasından yapılan değişiklikler anında workflow'a yansır.

4. Route By Type — Switch Node

Kaynakları tipine göre 3 farklı kola yönlendirir:

- **Çıkış 0:** `type === 'rss'` veya `'youtube'` veya `'podcast'` → RSS yoluna gider
- **Çıkış 1:** `type === 'json'` → JSON API yoluna gider
- **Çıkış 2:** `type === 'html'` → HTML scraping yoluna gider

Source eklerken type'ı nasıl bileceğim?

Type	Ne zaman seçilir	Örnek
rss	Site'nin /rss , /feed , /atom.xml gibi bir endpoint'i varsa	FAA Newsroom, EASA News
youtube	YouTube kanal feed'i	youtube.com/feeds/videos.xml?channel_id=...
json	Site direkt JSON dönen bir API endpoint'i sunuyorsa	Özel API'ler
html	Yukarıdakilerin hiçbiri yoksa — normal web sayfası	AVHerald, SKYbrary, NTSB

Nasıl kontrol ederim?

1. Tarayıcıda site.com/rss veya site.com/feed dene
2. Eğer XML içerik açılıyorsa → rss
3. Açılmıyorsa → html

5. RSS Read — RSS Feed Read Node

RSS/Atom feed URL'ini okur, her makaleyi ayrı bir item olarak çıkarır. YouTube da RSS formatında feed sunduğu için buradan geçer.

6. Normalize RSS — Code Node

RSS'ten gelen ham veriyi standart formata çevirir:

```
{
  source_id, source_name, source_type: 'rss',
  title, url, content, summary,
  published_at, fetched_at
}
```

7. HTTP JSON — HTTP Request Node

JSON API tipindeki kaynakların URL'ini GET ile çeker. Ham JSON döner.

8. Normalize JSON — Code Node

JSON API'den gelen veriyi standart formata çevirir. items , articles , results , data , entries gibi yaygın key isimlerini otomatik tanır.

9. HTTP HTML — HTTP Request Node

HTML kaynak sitesinin ana URL'ini `text` formatında çeker. Tüm HTML string olarak gelir.

10. Extract Links — HTML Node

Ana sayfadan tüm `<a>` etiketlerini çıkarır:

- `titles` : Linkin görünen metni
- `links` : `href` değeri

Sonuç örneği:

```
titles: ["NTSB Investigation Report", "FAA Safety Alert", ...]
links: ["/investigations/2026-001", "/safety/alert-42", ...]
```

11. Pick Subpages — Code Node

Extract Links'ten gelen tüm linkler arasından **anlamalı olanları filtreler**.

Hangi linkleri seçer?

Şu regex ile eşleşenleri alır:

```
/news|article|press|release|directive|airworthiness|bad\b|safety|notice|bulletin|report|incident|advisory|alert/i
```

Hangileri atlar?

```
/\/(about|contact|privacy|terms|login|register|cookie|sitemap)\b/i
```

Her kaynaktan **maksimum 15 link** seçer.

⚠ AVHerald sorunu: AVHerald bazı ülkelerden veya IP'lerden erişimi engelliyor (Cloudflare koruması). Bu yüzden içerik çekilemiyor veya boş geliyor.

12. HTTP Subpage — HTTP Request Node

Seçilen her alt sayfayı **3'erli batch** halinde HTML olarak çeker.

`batchSize: 3` → Sunucuyu aşırı yüklememek için rate limiting.

13. Extract Content — HTML Node

Her alt sayfadan 3 şey çıkarır:

- `page_title` : `<title>` etiketi
- `h1` : Sayfa ana başlığı
- `body_text` : Tüm `<body>` içeriği (ham metin)

14. Prep Text — Code Node

Çıkarılan metni birleştirir ve **6000 karakterle keser** (GPT token limitini aşmamak için):

```
const parts = [j.h1, j.page_title, j.body_text].join('\n');
return { page_text: String(parts).slice(0, 6000) };
```

15. Classify Subpage — Information Extractor (LangChain)

★ **En önemli node — Category ve Severity buradan geliyor!**

GPT-4.1-nano'ya sayfa metnini gönderir ve şu alanları çıkarmasını ister:

Alan	Açıklama
is_relevant	Bu sayfa gerçek bir haber/makale/AD mı? (true/false)
content_type	news , article , ad , other
title	Makalenin başlığı
summary	1-2 cümle özet
published_at	ISO 8601 formatında yayın tarihi
category	Airworthiness Directive , Airbus , Boeing , Safety , General

⚠ **Severity yok!** Şu an workflow'da severity alanı hiç hesaplanmıyor. Sitede gördüğün severity değerleri Next.js tarafında category'den türetiliyor:

```
Airworthiness Directive → HIGH
Safety → MEDIUM
Boeing / Airbus → LOW
General → INFO
```

16. OpenAI Chat Model — LLM Node

Classify Subpage node'una bağlı language model. `gpt-4.1-nano` kullanıyor (ucuz ve hızlı). Bu node doğrudan çalıştırılmaz, Classify Subpage'in "beyin"i olarak bağlıdır.

17. Filter Relevant — Code Node

GPT'nin `is_relevant: true` dediği sayfaları geçirir, false olanları atar. Index sayfaları, login sayfaları, menüler vs. burada elenir.

18. Normalize HTML — Code Node

HTML akışını standart formata çevirir. GPT'nin çıkardığı `title`, `summary`, `published_at` alanlarını kullanır.

19. Merge Sources — Merge Node

RSS, JSON ve HTML kollarından gelen tüm normalize edilmiş içerikleri tek bir akışta birleştirir (3 giriş → 1 çıkış).

20. Filter Valid — Code Node

`title` ve `url` alanı boş olan kayıtları atar. Minimum kalite filtresi.

21. Add Hash — Code Node ★

Kategori ve Tag'ler BURADA belirlenir (RSS ve JSON için).

Metin içinde keyword arar:

```
if (/airbus/.test(text)) tags.push('Airbus');
if (/boeing/.test(text)) tags.push('Boeing');
if (/airworthiness directive|emergency ad|bad\b/.test(text)) tags.push('AD');
if (/safety|incident|accident|emergency|inspection/.test(text)) tags.push('Safety');
```

Öncelik sırası:

```
AD var mı? → category: 'Airworthiness Directive'
Airbus var mı? → category: 'Airbus'
Boeing var mı? → category: 'Boeing'
Safety var mı? → category: 'Safety'
Hiçbiri? → category: 'General'
```

⚠ **Sorun:** Bu çok basit keyword matching. “Airbus’ın Boeing’e rakip olması” gibi bir haber hem Airbus hem Boeing tag’i alır ama Airbus olarak kategorize edilir. Daha akıllıca bir çözüm için GPT sınıflandırmasını RSS/JSON akışına da eklemek gerekir.

Ayrıca `content_hash` üretir — her belgenin parmak izi:

```
hash(title_normalized + '|' + url_normalized)
```

22. MongoDB Find Existing — MongoDB Node

Mevcut `aviation_news` collection’ından son 1000 kaydın `content_hash`, `url` ve `embedding` alanlarını çeker. Duplicate kontrolü için kullanılır.

23. Hash Dedup — Code Node

Yeni gelen kayıtlarla mevcut kayıtları karşılaştırır:

1. `content_hash` aynıysa → atla
2. `url` aynıysa → atla
3. İkisi de farklıysa → devam et

24. OpenAI Embedding — HTTP Request Node

Her yeni kayıt için `text-embedding-3-small` modeli ile vektör üretir.

Input: `title + " " + summary` (max 2000 karakter)

Output: 1536 boyutlu float array

Bu vektörler daha sonra semantic search ve AI chat için kullanılır.

25. AI Semantic Dedup — Code Node

Hash'ten kaçan duplicate'leri vektör benzerliğiyle yakalar.

Cosine similarity eşiği: 0.90 — %90'dan fazla benzer iki içerik duplicate sayılır.

Örnek: "FAA Issues Safety Alert" ile "FAA Releases Safety Notice" → hash farklı ama anlam aynı → biri atılır.

26. Build Documents — Code Node

MongoDB'ye yazılacak final belgeyi oluşturur. Tüm alanları normalize eder, eksik alanları doldurur.

27. MongoDB Insert — MongoDB Node

`aviation_news` collection'ına yeni kayıtları ekler. Şu alanlar yazılır:

`source_id`, `source_name`, `source_type`, `title`, `title_normalized`, `url`, `content`, `summary`, `published_at`, `fetches_at`, `content_hash`, `embedding`, `category`, `tags`, `dedup_group_id`

Sorularına Cevaplar

? Neye göre Severity veriyor?

Şu an workflow'da severity yok. Sitede gördüğün severity tamamen Next.js frontend'de `category` 'den türetiliyor:

```
Airworthiness Directive → HIGH (turuncu)
Safety → MEDIUM (sarı)
Boeing / Airbus → LOW (yeşil)
General → INFO (mavi)
```

Önerim: `Classify Subpage` node'una `severity` alanı ekle (GPT karar versin):

CRITICAL: Fatal accident, hull loss
 HIGH: Airworthiness directive, emergency
 MEDIUM: Incident, safety bulletin
 LOW: Advisory, informational
 INFO: News, general

? Neye göre Category veriyor? Yanlılık var mı?

Evet, potansiyel sorun var. İki farklı noktada belirleniyor:

- **HTML kaynaklar** → `Classify Subpage` (GPT) belirliyor
- **RSS/JSON kaynaklar** → `Add Hash` (keyword regex) belirliyor

Keyword regex çok basit. Örneğin Simple Flying'deki "How Airbus Noses Differ From Boeing's" haberi hem Airbus hem Boeing içerdiğinden Airbus olarak kategorize ediliyor ama aslında "General" olmalı.

Önerim: RSS/JSON akışına da bir GPT classification node'u ekle. Böylece tüm kaynaklar aynı kalitede kategorize edilir.

? Detayda daha fazla içerik istiyorum

Şu an `content` alanına sadece GPT'nin ürettiği 1-2 cümlelik `summary` yazılıyor:

```
content: o.summary || '', // sadece özet!
```

Sorun: Orijinal sayfa içeriği (`body_text`) MongoDB'ye kaydedilmiyor, sadece özet var.

Çözüm: `Normalize HTML` node'unda `body_text` 'i de kaydet:

```
content: o.full_content || body_text || o.summary || '',
```

Bunun için `Classify Subpage` 'e `full_content` alanı eklenebilir (GPT'nin temizlenmiş tam metin döndürmesi için).

? AVHerald siteye özel scraping mümkün mü?

Evet, çok mantıklı. AVHerald'ın URL ve HTML yapısı tahmin edilebilir:

```
https://avherald.com/h?article=XXXXXXXXX → her haber unique URL
https://avherald.com/h?list= → liste sayfası
```

AVHerald haberlerinde şu yapı var:

- `h1` → Olay başlığı (Incident/Accident: [uçak tipi] [nerede] [ne oldu])
- `.h5` → Detay paragrafları
- Başlık formatı: `Incident:` veya `Accident:` ile başlar → otomatik severity tespiti için kullanılabilir:
- `Accident:` → CRITICAL veya HIGH
- `Incident:` → MEDIUM veya HIGH

- bird strike , gear , pressurization → MEDIUM
- engine failure , fire → HIGH

Bu pattern'i Pick Subpages veya yeni bir "AVHerald Parser" node'unda implement edebiliriz.

? Source eklerken Type'ı nasıl bileceğim?

Hızlı kontrol yöntemi:

1. RSS var mı? Tarayıcıda dene:

- site.com/rss
- site.com/feed
- site.com/atom.xml
- site.com/news.xml
- Eğer XML içerik açılıyorsa → rss

2. YouTube kanalı mı? URL youtube.com içeriyorsa → youtube

- Kanal feed URL formatı: `https://www.youtube.com/feeds/videos.xml?channel_id=UC...`
- Kanal ID'sini bulmak: YouTube kanalına gir → sağ tık → "Sayfa Kaynağını Gör" → channel_id ara

3. JSON API? → Genellikle sadece özel/kurumsal sistemlerde olur. `site.com/api/news` gibi.

4. Hiçbiri yoksa → html

Pratik kural: Düzenli olarak güncellenen kurum siteleri (NTSB, ICAO, SKYbrary) genellikle RSS sunar. Haber siteleri (Simple Flying, AVHerald) genellikle HTML.

Önerilen Geliştirmeler (Öncelik Sırası)

#	Geliştirme	Efor	Etki
1	Load Sources → MongoDB'den oku	Orta	Yüksek — site ile senkronize olur
2	Severity alanı ekle (GPT)	Düşük	Yüksek — sitenin anlam kazanması
3	body_text 'i content olarak kaydet	Düşük	Yüksek — detay sayfası zenginleşir
4	RSS/JSON akışına da GPT classification ekle	Orta	Orta — tutarlı kategori
5	AVHerald özel parser	Orta	Orta — blok aşma + zengin içerik
6	Rate limiting / retry mekanizması	Yüksek	Orta — daha az hata